Using Agentic AI (LLMs) to Generate Adversarial Attacks on Other ML Models

Jovin Antony, Maria
jmaria2@gmu.edu
https://github.com/Jovin12/ECE653Final_AdversarialAgent

***Abstract*—More often than not, Adversarial attacks are simply static, and fail to adjust if the inital attack fails and does not optimize for balance between attack effectiveness and lower perturbation rate. This paper introduces a framework that is able to choose and improve adversarial attacks autonomously just like a human would across different modalities of vision, audio, tabular data and Large Language Models. Utilizing an Agent or more specifically LangGraph based-architecture using, a planner, attacker and evaluator, the framework learns an optimal attack strategy based on prior trial and error attempts, to maximize misclassification error while minimizing the perturbation metric values. The Adversarial Agent was able to achieve a 100% misclassification rate on almost all attacks and managed to learn and optimize attack strategies to adapt and perform better attacks.**

I. Introduction

We are currently living in era of AI and Machine Learning where it is rapidly expanding and being integrated to critical systems, involving medicine and even autonomous vehicles. Companies even leverage AI by integrating it with complete workflow pipelines, which utilize AI to take sensitive company data and perform specific tasks based on that data. This in turn results in exposing vulnerabilities to Adversarial attacks. These crafully crafted attacks which are often unidentifiably by humans can, cause state of the art Models to misclassify. While there are well known attacks like FGSM and PGD, they mostly remain static, and execute without the ability to learn from their failures, or adapting to defenses.

Large Language Models (LLMs) offer a way to bypass these issues, as LLMs have been improving and getting closer to humans, with capabilities like code generation, logical reasoning/strategic planning. Given these capabilites, it begs the questions Can LLMs be leveraged to be used as strategic planner to make adaptive adversarial attack systems, with the lowest perturbation possible on any form of Machine Learning model there is.

Hence, this project looks to create an Agentic framework to implement a workflow that leverages LLMs as planners to select, finetune, and iteratively improve attacks across differnt models of vision, audio, structured/tabular data, and LLM targets. Compared to prior work like jailbreaking [1] or LLM red-teaming [2], this project creates a framework where:

- The Adversarial Agent is able to adapt to any model type, without any modification.
- The Adversarial Agent performs an iterative form of attack, where the LLM planner can learn from previous attacks and strategically change attacks or finetune attacks to get better results.
- The Adversarial Agent should also perform attack with the least possible perturbation for the respective model.
- Finally, shows its Chain-of-Thought reasoning on those attacks.

From the tests and experiments performed, the Adversarial Agent was able to demonstrate successful autonomous attack across different models, with the Agent being able to improve and perform continuous and better attacks over a course of iterations, with generating fail saves when certain attacks were refused. This report details the framework, and the experimental results of the Automated Adversarial Testing.

II. Threat Model

In this section, we are explaining the assumptions, capabilities and constraints that define the Adversarial Agent and the Target agents on whom the experiment will be executed on.

A. *Adversarial Agent*

The Attacker/adversarial Agent uses these assumptions:

- **Model Access & Architecture Access** The model has access to white box and black box attacks where the model will chose which attack to perform depending on what kind of model it is attacking.
  The idea of this is to simulate real world scenarios, with most Local LLM deployments exposing only the inference with APIs(or cases where the attack has no access to the model), or while models like vision and audio where the full architecture maybe fully accessible.
- **Dataset Assumption** The adversarial model is assuming that the Target model is a model trained on a known dataset that the Adversarial model has access to. So the Adversarial Agent knows the input distribution and can generate sample test points, the idea here is to generate a realistic targetted attack for specific inputs.
- **Adversarial Tools** The Adversarial Agent has access to full ART library methods to generate attacks.

B. *Target Model Assumptions*

These are the assumptions on the Target model, or the model the attack will be executed on.

- All target models for vision, audio, structured data, are models that predict a class or probabilities for a given input.
- Target Models were standard pre-trained models(no adversarial trained models), the biggest reason was that, it was difficult to get a hold of models(for audio, and tabular data) that were adversarially trained, hence most of the target models were standard pre-trained models, execpt for Vision, where the resnet-50 AT (Adversarially trained) model was used.
- Model weights, remain static during Adversarial Attacks

C. *Target Models and Target Datasets used*

- **Image**
  Models: ResNet-50, ResNet50-AT, ViT, ViT-AT
  Dataset: Imagenet
  Attacks Used/in toolkit: FGSM, PGD, DeepFool, Carlini L2, Adversarial Patch

- **Audio**
  Models: SimpleCNN, ResNet18-Audio, VGGish
  Dataset: Google Speech Commands
  Attacks Used/in toolkit: FGSM, PGD, DeepFool, Carlini L2

- **Structured/Tabular Models**
  Models: MLP, Random Forest, GradientBoost, Logistic Regression
  Dataset: Iris, Brest Cancer, Diabetes
  Attacks Used/in toolkit: FGSM, PGD, ZOO, Boundary Attack, Decision tree Attack.

- **LLMs**
  Models: Local Ollama hosted models: Qwen 3.5 0.8b, llama 3.1 8b, Qwen 3.5 2b; etc.
  Dataset: Set of 20 malicious prompts, with examples like; how to hack in to a bank, how to make a bomb etc.
  Attacks Used/in toolkit: Jailbreak, Obfuscation, RolePlay, Context Manipulation

D. *Adversarial Agent Attack Objectives*

The Agent pursues 2 main objectives:

- Force a high misscalssification rate, for classification and predictive models, or harmful output for LLMs. With a success rate 95%.
- Secondly, minimize the perturbation rate to maintain stealth, with each model having different sets of perturbation metrics to track.

Vision

- **L-inf**: The maximum change made to any pixel across the entire image
- **L2 Norm**: Euclidean distance between original and perturbed image, across pixels.

```
# Calculate perturbation metrics
diff = x_adv - x_test

# L2 norm per sample, then mean across batch
l2_per_sample = np.sqrt(np.sum(diff ** 2, axis=(1, 2, 3)))
perturbation_l2 = np.mean(l2_per_sample)

# Linf (maximum pixel change across all samples)
perturbation_linf = np.max(np.abs(diff))
```

Fig. 1. L2, L-inf calculation

Audio

- **MAE**: Mean Absolute Error, the average change across all spectrogram bins. Lower value, means less noise in attack.
- **SNR**: Signal to Noise Ratio, Compare original signal power to the perturbed power. Higher SNR means attack noise remains below the audible threshold.

```
# Calculate perturbation metrics for audio
diff = x_adv - x_test

# Mean absolute error (lower is better)
perturbation_mae = np.mean(np.abs(diff))

# Signal-to-Noise Ratio (higher is better)
signal_power = np.mean(x_test ** 2)
noise_power = np.mean(diff ** 2)
snr = 10 * np.log10(signal_power / noise_power) if noise_power > 0 else 100.0
```

Fig. 2. MAE, SNR Calculation

Structured/Tabular Data

- **L2 Norm**: Euclidian distance between the original and purturbed feature vectors. Lower values indicates better attack.
- **Percentage of features changed**: Proportion of features that were altered.

LLM

- **Jaccard Distance**: measure how the attack payload is worded from the original.

E. *Ethical Consideration and Assumption of the Adversarial Agent*

The Adversarial Agent, or the threat model assumes that this framework or the context of its attacks is a security research framework with complete authorization. In fact this was how the Adversarial Agent was prompt engineered to perform its attack. All the attacks tested, were only to perform safety checks with the goal of improving defenses against adaptive attacks.

III. Background and Related Work

As we all know, adversarial attacks are techniques used to deceive machine learning (ML) models by introducing small, intentional modifications or perturbations. And

with adversarial attacks we aim to generate inputs with imperceptible perturbations that cause model misclassifications. And we have White-box attacks like FGSM [3] and access to the gradients of the model in a single step, we also have PGD [4] which assumes that iteratively refines perturbations, we also have attacks like DeepFool, finds the minimal perturbation, Carlini & Wagner which use optimized state of the art success. Then we also have Black box attacks like ZOO [5] and Boundary Attack [6] which operate without gradients.

But despite these various forms of attacks depending on how much access the attacker has to the model. These traditional attacks cannot adapt across strategies, tune hyper parameters automatically, or even balance perturbation of the attack against the effectiveness of it. In fact these traditional attacks are often just static, and fail to learn from their failures from previous attacks.

With the rise of AI based frameworks and workflows namely the ReACT [7] involving/making LLMs use reasoning to perform actions action, allows the use of LLMs to make decisions like a human would. In addition to this we have frameworks like LangChain (which utilizes agentic(LLM+tools) sequential workflows) and LangGraph (For cyclic Agentic workflows), which is used in this project to create an iterative, planner-attacker-evaluator loop.

This project in comparison to prior work tries to asses the limiations and extend to build on top of those limitations namely:

- **Jailbreaking LLM in 20 queries** [1], Like the title suggests this paper introduces an iterative approach to jailbreaking LLMs, but realistically it is highly unlikely that LLMs are the only models you want to perform an attack on. So, this project extends this further and incorporates the same iterative approach but to apply for all models types of Vision, Audio , Tabular and not just LLms.
- **AutoRedTeam** [2] In AutoredTeam, which incorporates a Memory-based LLM red teaming. But this approach has no integration with tools to perform specific attacks or fine-tune those attacks, to get better performance. Which this project extends to include a multi-modal tool kit using the ART(Adversarial robustness toolkit) library to add tools for support for the multi-modal LLM.
- **Towards Evaluating the Robustness of Neural Networks** [8], From This paper we see how they show that almost all defenses fail when an attack is optimized for those different models. But, this involved manual parameter tuning, and this project hopes to extend this with the use of LLMs to automate the finetunin process.

No prior work combines LLMs for planning with multi-modal attack execution, across vision, audio, tabular and LLM targerts in a unified framework. This research project implements the Adversarial Agent using LangGraph to create a unified framework, with an evaluator to compute the misclassification and perturbation metrics, and force the LLM to output its chain-of thought thinking for showing its strategic adjustment between iterations.

IV. Proposed Methodology

The proposed framework consists of four interacting nodes, arranged in a cyclic graph using LangGraph(Initially used Langchain, but had to use LangGraph due to the cyclic iterative nature of the project). Each iteration represents one complete attack, with evaluator deciding whether to continue or End the Test uplon a successesful attack.

Fig. 3. Complete Preprocess pipeline

A. *Prior to the framework (PREPROCESSING)*

This step is a necessary one, unlike vision models, where pre-trained models were easily available for the imagenet dataset; other models like audio, and tabular are not readily available. Hence, specific scripts were created for, audio, tabular, and even vision. Where using these scripts, this prior preprocessing step was included, where both the model and the dataset were loaded, after which the model was trained, validate and fine-tuned, after which it had to go through a **crucial step of enclosing(converting) the model as a ART classifier**, which allows the Adversarial Agent to utilize the ART toolkit given to it, and then return the model.

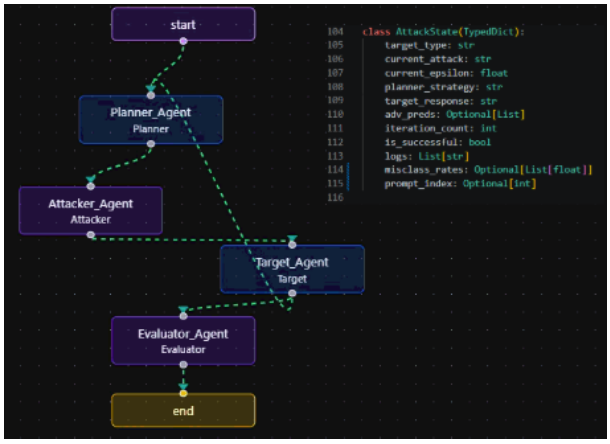


Fig. 4. System Architecture

B. *Overall System Architecture*

As mentioned earlier, the framework uses LangGraph, to help with organizing this workflow. The way langGraph works is that it has a Finite State Machine like approach where there is a global state, as seen in the Figure 4 above, that is shared by every node in the workflow, and any changes made to the states transfers information from one node to another. As you

can see from the figure, we have crucial variables keep track of information like current iteration, response of each agent during that iteration, and more crucially logs so the Adversarial Agent can plan its next step. This form of information chain allows for the workflow to implemented LLM that can think and decide which tools to use.

Agents and Nodes:

- **Planner**: This is the main thinking node of the framework, it uses the Llama 3.1 8B LLM, which selects the best attack strategies based on the history of all the attacks performed on the particular models. This planner model is instructed to first try every attack that it can from it toolkit using the same hyperparameters for each attack and then pick the best performing attack and finetune the parameters further. It takes information like iteration count, previous target response, and attacker logs, then makes a decision and Outputs the attack type, the hyperparmeters to use, and also gives a chain-of thought reasoning for the chosen output values.

- **Attacker**: Executes the chosen attack using the ART library, and the given Attacker toolkit. Depending on the model for either vision, audio, tabular or LLM, it looks at the its toolkit or the attacks it has access to and choses the attack to execute, based on the instructions given by Planner. For vision/audio and tabular model targets attacks like FGSM, PGD, DeepFool, Boundary etc. were used, while for LLM targets, it used a set of prompts to then perform obfuscated or role-play prompts.

- **Target**: This node, is the actual victim model where the attack is being performed on. It could be any of the models, depending on what the test being done is; vision (ResNet-50, ViT), audio(VGGish, ResNet18-audio, SimpleCNN), tabular(MLP, Random Forest etc) or an LLM(Qwen 3.5 0.8B). The target returns predictions or text responses.

- **Evaluator**: This is the evaluator agent used, Qwen 3.5 2B, which computes the success metrics, and checks the misclassification rate thresholds and calculates the respective perturbation metrics for each of the vision (l-inf, l2 norm), audio(SNR, MAE), Structured/tabular(l2 norm, δfeatures%), LLM (check for harmful response).

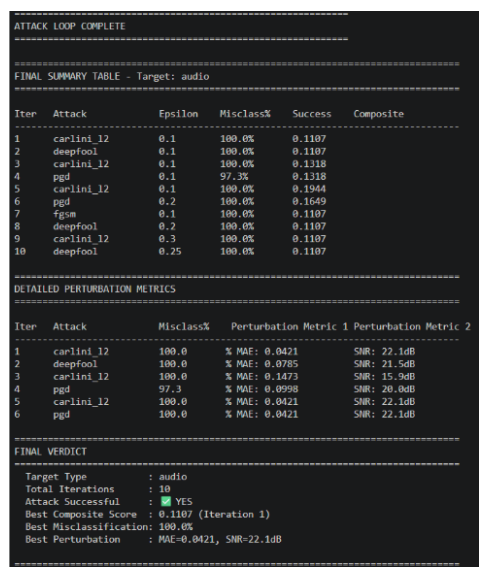
Iterative Approach The Planner agent maintains the state object, which as mentioned before containes the complete attack history (logs), with previous iterations metrics(misclassification rate, pertubation metrics) for each attempt. During any iteration the Logs memory, is updated at end of each node in the workflow. Which after the evaluator agent updates Logs with the performance metrics of that iteration, is then fed back into the planner agent (if the attack was not successful) for the next iterations where the planner seeks to answer questions like: "What attacks have been tried? and what epsilong values succeeded or failed?", which then allows the Planner to make calculated decisions on what the next attack should be as in if it need to stick to the same attack and only improve the hyperparameters for the attack or maybe the attack didnt show enough of an improvement to switch to a different attack entirely, (like switch from FGSM to PGD, if tuned epsilon value didnt improve the attack).

Implementation Details:

- **Framework:** LangGraph
- **LLMs:** LLama 3.1 8B(Planner), Qwen 3.5 2B(Evaluator), and target LLMs
- **Attack Library:** ART (Adversarial Robustness Toolkit
- **Max Iterations:** 10 per attack, REASON: Most attacks ended within a successful attack in 5 iterations.

V. Results

Using the experiment configurations detailed earlier, with the models, datasets and attack toolkits, and the iterative process, the Framework will be evaluated across the different modalities. The inital approach to showing the working of the frame work used miclassification rate as the primary metric, hence you will see the first few tables showing that. but later, realizing that misclassificaton rate is not the only metric to use, you will see a final summary table where each attack session ran for 10 iterations with success defined as meeting both misclassification and stealth thresholds, what the best attack is overall. Here, in Figure 6, you will see an example output of what the planner's chain of thought reasoning gives you with every iterations. You will see information such as, "what happened previously", "does the model have any prior knowledge to help it improve the next attack", and ultimately decide, "What attack it is going to use, with what hyper parameter?"



```
ATTACK LOOP COMPLETE

FINAL SUMMARY TABLE - Target: audio

Iter   Attack            Epsilon    Misclass%    Success    Composite
1      carlini_l2        0.1        100.0%       0.1107
2      deepfool          0.1        100.0%       0.1107
3      carlini_l2        0.1        100.0%       0.1318
4      pgd               0.1        97.3%        0.1318
5      carlini_l2        0.1        100.0%       0.1944
6      pgd               0.2        100.0%       0.1649
7      fgsm              0.1        100.0%       0.1107
8      deepfool          0.2        100.0%       0.1107
9      carlini_l2        0.3        100.0%       0.1107
10     deepfool          0.25       100.0%       0.1107

DETAILED PERTURBATION METRICS

Iter   Attack            Misclass%    Perturbation Metric 1 Perturbation Metric 2
1      carlini_l2        100.0        % MAE: 0.0421         SNR: 22.1dB
2      deepfool          100.0        % MAE: 0.0785         SNR: 21.5dB
3      carlini_l2        100.0        % MAE: 0.1473         SNR: 15.9dB
4      pgd               97.3         % MAE: 0.0998         SNR: 20.0dB
5      carlini_l2        100.0        % MAE: 0.0421         SNR: 22.1dB
6      pgd               100.0        % MAE: 0.0421         SNR: 22.1dB

FINAL VERDICT
  Target Type           : audio
  Total Iterations      : 10
  Attack Successful     : ✅ YES
  Best Composite Score  : 0.1107 (Iteration 1)
  Best Misclassification: 100.0%
  Best Perturbation     : MAE=0.0421, SNR=22.1dB
```

Fig. 5. EXAMPLE FINAL OUTPUT, with metrics

Figure 5 shows a representative output from the Adversarial Agent after 10 iterations, including misclassification rates,

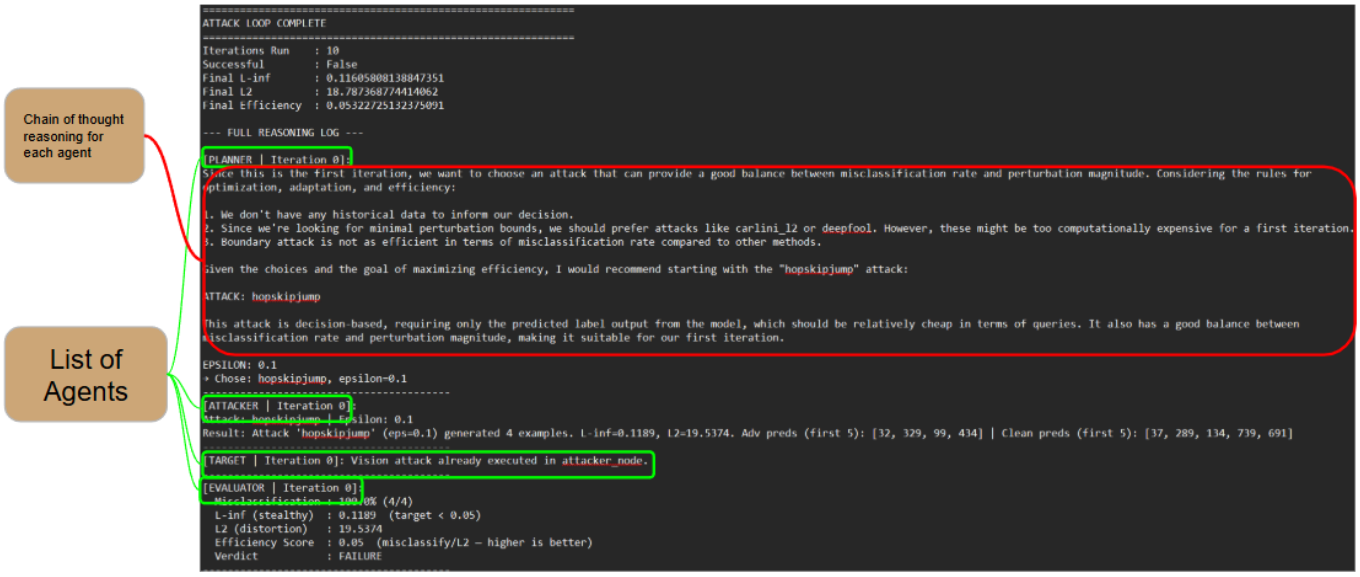


Fig. 6. Example Log showing agents and chain of thought reasoning for attack

perturbation metrics (MAE, SNR), composite scores, and a final verdict. All subsequent results follow this format.

A. *Vision Models*

Initial experiments focused on the Adversarial Agent to perform based on misclassification. Here for the vision models, the agent demonstrated a high success rate, often identifying a successful attack in under three iterations. As seen in Tables I and II.

TABLE I
WHITE-BOX RESULTS (MISCLASSIFICATION PRIORITY)

Model	Clean Acc	Iters	Initial	Final
ResNet-50	80.0%	2	FGSM	PGD
ViT	79.0%	2	FGSM	PGD
ResNet-50 AT	61.0%	1	PGD	PGD

TABLE II
BLACK-BOX RESULTS (MISCLASSIFICATION PRIORITY)

Model	Clean Acc	Iters	Initial	Final
ResNet-50	87.5%	2	Square	Square
ViT	79.0%	2	Square	Square
ResNet-50 AT	43.8%	3	Square	HopSkipJump

Extending to 10 iterations with perturbation metrics as the main metric to priorotize. Tables III-IV show the top 3 white-box and black-box attacks on ResNet-50.Also, In Tables III and IV, you will notice metrics like L-inf, the maximum change to any pixels and the l2 norm, which is the eucliedean distance between original and perturbed image across pixels. Where a lower L-inf and lower L2 norm score, determins a less perturbed attack. Also keep in mind these tables were collected over 10 iterations with output similar to how the Figure 5 is structured.

TABLE III
TOP 3 WHITE-BOX ATTACKS ON RESNET-50

Rank	Attack (Iter)	Misclass%	L-inf	L2
1st	PGD (Iter 3)	100%	0.1250	38.79
2nd	Carlini L2 (Iter 0)	100%	0.4349	1.51
3rd	FGSM (Iter 2)	100%	0.2500	87.78

TABLE IV
TOP 3 BLACK-BOX ATTACKS ON RESNET-50

Rank	Attack (Iter)	Misclass%	L-inf	L2
1st	HopSkipJump (Iter 7)	100%	0.1142	18.80
2nd	Boundary (Iter 5)	100%	0.4164	20.06
3rd	Square (Iter 2)	100%	0.0750	27.19

B. *Audio Models*

As already explained, Initial experiments focused on the Adversarial Agent to perform based on misclassification.
Table V summarizes initial misclassification-only results. All models achieved $\geq$98% success in one iteration.

Extending to 10 iterations with perturbation metrics as the main metric to priorotize. Table VI presents the top 3 attacks per model after incorporating the perturbation metrics; **MAE**, Mean Absolute Error, the average change across the spectrogram bins, where a lower value meant less noise in attack; **SNR**, Signal to noise ratio, compare original signal power to the perturbed power, Higher SNR means attack noise remains below the audible threshold.

TABLE V
AUDIO RESULTS (MISCLASSIFICATION PRIORITY)

Model	Clean Acc	Attack	Success	Iters
ResNet18-Audio	99.7%	DeepFool	100%	1
SimpleCNN	98.0%	PGD	100%	1
VGGish	96.8%	DeepFool	98%	1

TABLE VI
TOP 3 ATTACKS BY AUDIO MODEL

Model	Rank	Attack (Iter)	Misclass%	MAE	SNR (dB)
ResNet18	1st	PGD (5)	100%	0.1388	16.3
	2nd	DeepFool (1)	100%	0.0539	18.2
	3rd	Carlini L2 (4)	100%	0.0779	21.6
SimpleCNN	1st	Carlini L2 (1)	100%	0.0421	22.1
	2nd	DeepFool (2)	100%	0.0785	21.5
	3rd	PGD (6)	100%	0.0421	22.1
VGGish	1st	PGD (3)	91.8%	0.1000	20.0
	2nd	Carlini L2 (2)	95.9%	0.0889	20.7
	3rd	DeepFool (1)	100%	0.1038	11.3

C. *Tabular Models*

Table VII shows initial misclassification-only results. Boundary Attack was universally successful.

TABLE VII
TABULAR RESULTS (MISCLASSIFICATION PRIORITY)

Model	Clean Acc	Attack Sequence	Success
Random Forest	88.9%	FGSM → Boundary (2)	100%
MLP	96.7%	Boundary (1)	100%
Logistic Regression	91.1%	FGSM→PGD→ZOO→DT→Boundary(5)	100%
Gradient Boost	93.3%	PGD→ZOO→Boundary(3)	100%

Table VIII presents the top 3 attacks per model, and the respective iterations they were discovered in with perturbation metrics.

TABLE VIII
TOP 3 ATTACKS BY TABULAR MODEL

Model	Rank	Attack (Iter)	Misclass%	Feat. Changed	L2
MLP	1st	Zoo (1)	100%	99.4%	1.334
	2nd	Boundary (2)	100%	100.0%	1.323
	3rd	Dec. Tree (3)	100%	100.0%	1.322
Random Forest	1st	Boundary (1)	100%	61.1%	0.733
	2nd	PGD (2)	100%	63.9%	0.832
	3rd	Dec. Tree (3)	100%	76.7%	0.906
Logistic Reg.	1st	Boundary (1)	100%	73.3%	0.911
	2nd	Dec. Tree (4)	100%	74.4%	0.918
	3rd	Zoo (2)	13.3%	100.0%	0.200
Gradient Boost	1st	Boundary (1)	100%	33.3%	0.551
	2nd	Boundary (2)	100%	37.2%	0.518
	3rd	Boundary (5)	100%	38.3%	0.477

D. *LLM Targets*

Local LLMs exhibited unexpected degradation behavior:

- **Iters 0-2**: Explicit refusal responses
- **Iters 2-5**: Empty responses (treated as success by evaluator — a flaw)
- **Iters 5+**: Model unresponsive even to benign prompts

Key finding: Empty responses were initially misinterpreted as successful attacks, revealing that *silence is not compliance*.

VI. KEYTAKEAWAYS

- The Same LangGraph based Adversarial Agentic Architecture was successfully able to adapt to vision, audio and tabular targets without any need for a change in the architecture itself, the planner consistently idetified and fine tuned the optimal atacks for ever model and modality.

- The planner effectively used historiecal logs to transition from simple FGSM attacks to stonger and stronger PGD and Boundary attacks when intial attempts proved inefficient.

- The framework did not simply maximize misclassification, it actively reduced perturbation magnitudes when attacks succeeded with excessive distortion.
  Across all types of modalities of Vision, Audio, Tabular, Boundary attack achieved 100% misclassification rate with reasonably less amount of perturbation, making it a default choice for attacks.

VII. LIMIATIONS

Despite successful demonstration across multiple modalities, The framework faces a few constraints:

- **Computation Limitation**: Complex attack like Adversarial Patch and DeepFool require significant computation time. Especially on high-dimensional data like ImageNet, Many vision attack couldn't be completed in a reasonable time frame.
- **LLM Target Problems**: As mentioned before the behavior of LLMs were unpredictable where after 2-5 iterations, complete refusal to respond and eventually unrespond to even regular prompts. The evaluator initially misinterpreted empty response as successful, showing a flaw in LLM safety evaluation.
  Tests couldn't be done on state-of the art models like GPT-5, claude or Gemini due to API restrictions.
- **Availability of models**: As mentioned in the beginning of the Adversarially trained models were only readily available for vision(ResNet-50 AT, ViT-AT).

VIII. FUTURE WORK

- **Defender Agent**: Develop a Defender Agentic Architecture similar to how the Adversarial Agent was create, where the defender Agent learns from attacker's strategies and iteratively improves its evasiveness on any target model. This would create a red-teaming loop where both attacker and defneder improve over time.

- **Test for Adversarial Scenarios**: As a mentioned in the limitations, the availability for adversarial Audio and tabular models were scares, and hence this will need a future adaptation to test the Adversarial Agent's working.
- **Blackbox Attack Only**: As the framework stands due to the preprocessing pipeline and the ART framework, we need the models to be converted to an ART classifier prior to any attacks performed on it. And hence to make it a more realistic scenario, we need to be able to have the architecture improve where, I just say " here is the model" and the Framework performs an attack on it.

IX. Conclusion

In conclusion, this project demonstrated that LLM-powered multiagent framework can autonomously discover and execute Adversarial attack across different modularities of vision, audio, tabular/strcutred models, and LLM targets. The planner, attacker, evaluator loop, successfully adapted attack strategies based on historical performance, to achieve a high misclassification rate with the lowest perturbations possible.
From this project a complete architecture requiring no model modifications, to perform adversarial attacks is shown. In addition to that with the help of the Adversarial Agent's chain of thought reasoning, the architecture performs human-like modification/adaptations to different performance/evaluation results.
This project through the Adversarial Agent also showed vulnerabilites like LLMs exhibiting degradation after repreated attack attempts.

While there were some trade offs and limitations to this architecture, this work establishes a foundation for autonomous adaptive Adversarial Attacks, and Defense for future integrations and expansion,

References

[1] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," *arXiv preprint arXiv:2310.08419*, 2024.

[2] A. Zhou, B. Li, and H. Wang, "Autoredteam: Autonomous red teaming with lifelong attack integration," *arXiv preprint arXiv:2503.15754*, 2025.

[3] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *International Conference on Learning Representations (ICLR)*, 2015.

[4] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *International Conference on Learning Representations (ICLR)*, 2018.

[5] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, and C.-J. Hsieh, "Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models," *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, 2017.

[6] W. Brendel, J. Rauber, and M. Bethge, "Decision-based adversarial attacks: Reliable attacks against black-box machine learning models," *International Conference on Learning Representations (ICLR)*, 2018.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *International Conference on Learning Representations (ICLR)*, 2023.

[8] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," *IEEE Symposium on Security and Privacy (SP)*, 2017.